

GITHUB BLOG · 新手指南

GitHub 新手上路 掌握核心要领的路线图

从版本控制到协作、自动化与开源——一条循序渐进、从零到自如的学习路径

 GitHub for Beginners · Polly Davidson

路线图

四步走，从零到自如使用 GitHub

这篇指南把整个 GitHub 新手系列串成一条路径，我们也照着它一站一站往前走。

01 打好基础

版本控制与 Git、账号安全设置、必备的 Git 命令

02 建立第一个项目

创建仓库、Markdown 排版、GitHub 工作流

03 与他人协作

拉取请求、合并与冲突、议题与项目板

04 更上一层楼

Actions 自动化、Pages 托管、代码安全、参与开源

PART 01

打好基础

版本控制、账号安全，与你每天都会用到的 Git 命令

版本控制

版本控制是什么？Git 又是什么？

版本控制会记录文件**随时间的每一次改动**，让你随时回溯——从此告别一堆 `final_v2`。



Git

在**本地**运行的工具，世界上用得最广的版本控制系统。



GitHub

托管 Git 仓库、支撑团队**协作**的云平台。

Git 的三个区域

 **工作区** working

↓ `git add`

 **暂存区** staging

↓ `git commit`

 **本地仓库** local repo

随时用 `git status` 看自己在哪一步

注册账号，先把安全设置做对



开启两步验证（2FA）

- ◆ 路径： [Settings → Password and authentication](#)
- ◆ 把恢复码存进**密码管理器**，别只截图
- ◆ 多一道验证，账号被盗风险大幅下降



打造个人资料 README

- ◆ 新建一个**与用户名同名**的仓库
- ◆ 其中的 [README](#) 会展示在你的主页顶部
- ◆ 用它做自我介绍，是你的第一张名片

安全设置只需做一次，却能保护你在 GitHub 上的每一行代码。

必备命令

十个几乎天天都会用到的 Git 命令

命令	作用
<code>git config --global user.name</code>	设置提交身份
<code>git init</code>	初始化一个新仓库
<code>git clone <url></code>	把远程仓库克隆到本地
<code>git status</code>	查看改动与暂存状态
<code>git add .</code>	暂存改动
<code>git commit -m "message"</code>	保存一次快照
<code>git switch -c <branch></code>	新建并切换到分支
<code>git push</code>	把提交上传到 GitHub
<code>git pull</code>	拉取最新的改动
<code>git merge <branch></code>	把分支合并到一起

PART 02

建立你的第一个项目

创建仓库、用 Markdown 排版，走通 GitHub 工作流

仓库

创建你的第一个仓库

仓库（`repo`）是一个会追踪改动、保存历史、让多人无缝协作的项目文件夹。

四步建好

- 1 从主页点击绿色的 `New` 按钮
- 2 给仓库起个名字
- 3 选择 **public** 或 **private**
- 4 勾选添加 `README`

`README`

项目的门面，来访者第一眼看到的说明。

`.gitignore`

列出 Git 应忽略的文件：系统文件、依赖、临时构建产物。

`license` （可选）

声明别人能怎样使用你的代码。

MARKDOWN

用 Markdown 轻松排版

Markdown 是一种**轻量的纯文本格式化语言**——几个符号，就能排出清爽的版式。

在 GitHub 上几乎无处不在，同一套写法通用：

[README](#)

[Issue](#)

[Pull Request](#)

[评论](#)

一级标题

****粗体**** **斜体**

- 无序列表项

1. 有序列表项

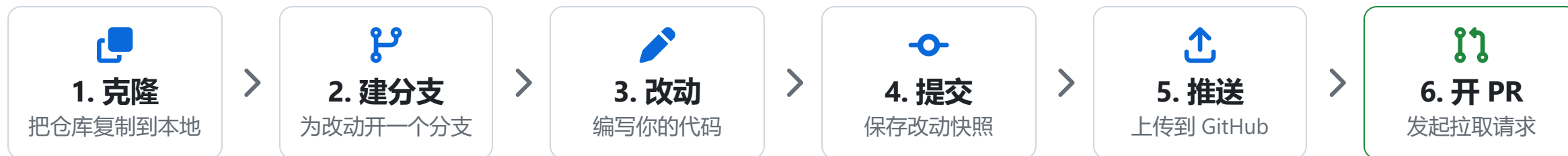
`行内代码`

[链接文字](https://github.com)

![图片说明](preview.png)

GitHub 工作流：一套可反复使用的协作循环

日常协作几乎都在重复这六个步骤——熟练之后会变成肌肉记忆。



给分支起个能说明意图的名字，比如 `fix-login-bug` 或 `add-dark-mode`。

PART 03

与他人协作

拉取请求、合并与冲突，以及议题和项目板

拉取请求

Pull Request: 把改动摆上台面讨论

拉取请求（[PR](#)）是把一个分支的改动**合并进另一个分支的提案**，并自带一块供队友评审、讨论的空间。

👁 直观展示改动的 diff 对比

💬 支持同行评审与逐行讨论

🍃 PR 越小，越好评审、越低风险

🔗 [add-dark-mode](#) → [main](#)

```
function toggleTheme() {  
-   setTheme('light')  
+   setTheme(saved ?? 'dark')  
}
```

合并与冲突

合并，以及如何化解冲突



多数合并，一键完成

PR 审查通过后，点一下 `Merge pull request`，改动就进了主分支。



当两个分支改了同一行

Git 无法替你决定，就会标出**冲突**：

```
<<<<<<< HEAD
setTheme('light')
=====
setTheme('dark')
>>>>>>> add-dark-mode
```

解决三步：在浏览器或 `VS Code` 里选定保留哪一版 → 标记 resolved → 合并。

用 Issues 和 Projects 管理工作



Issues 议题

- ◆ 追踪任务、bug 与想法
- ◆ 可**指派**给人、打**标签**分类
- ◆ 每条讨论都留有记录



Projects 项目板

- ◆ 把议题排到**看板**上
- ◆ 待办 / 进行中 / 完成，状态一目了然
- ◆ 团队进度尽在掌握



自动联动：在 PR 描述里写下关闭关键词 `Closes #42`、`Fixes #42` 或 `Resolves #42`，合并时会自动关联并关闭对应议题。

PART 04

让项目更上一层楼

自动化、免费托管、代码安全，与迈向开源

用 GitHub Actions 自动化重复工作

GitHub Actions 是内建于 GitHub 的 **CI/CD 与自动化平台**。

当仓库里发生事件时，自动帮你跑测试、部署、打标签等任务。

- ◆ 工作流写成 YAML，放在 `.github/workflows/`
- ◆ 指定**触发事件**与要执行的**步骤**
- ◆ 把重复的活儿交给机器

```
name: CI
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm test
```


GITHUB PAGES

用 GitHub Pages 免费托管网站

- ◆ 静态网站**免费**托管，无需服务器
- ◆ 在 [Settings → Pages](#) 一键开启
- ◆ 私有仓库也能用（输出站点是公开的）
- ◆ 作品集、项目文档的理想之选



[username.github.io/repo-name](#)

代码安全

内建的三道安全防线

GitHub Advanced Security 的这些能力对**公共仓库免费**，在仓库设置里即可开启。



Secret scanning

揪出误提交的 API 密钥与凭证，防止泄露。



Dependabot

监控依赖中的漏洞，自动开出升级 PR。



CodeQL

分析代码的数据流，发现潜在的危险模式。

安全不该是事后补救——让平台在每次提交时替你把关。

参与开源

迈出参与开源的第一步

开源软件的代码人人可读、可改进，而 GitHub 正是它的家。

怎么找到合适的项目

- ◆ 看是否有清晰的 `README`、`CONTRIBUTING.md` 和开源 license
- ◆ 寻找标着 `good first issue` 的议题

Branch 与 Fork 有何不同

Branch 分支

同一个仓库内部的平行工作区，你有权直接编辑。

Fork 派生

别人仓库的个人副本——给开源项目贡献时用它。

典型流程 >  Fork 项目 >  在 fork 里建分支 >  向原仓库开 PR

回顾

你的 GitHub 路线图，一图收尾



打好基础

Git 与版本控制、账号安全、必备命令



第一个项目

仓库、Markdown、GitHub 工作流



与他人协作

PR、合并冲突、议题与项目板



更上一层楼

Actions、Pages、代码安全、开源

GitHub 把版本控制、协作、自动化、托管与安全整合在同一个平台上。路线图已经在手中——现在，去建点什么的吧。



从一个仓库、一次提交开始。